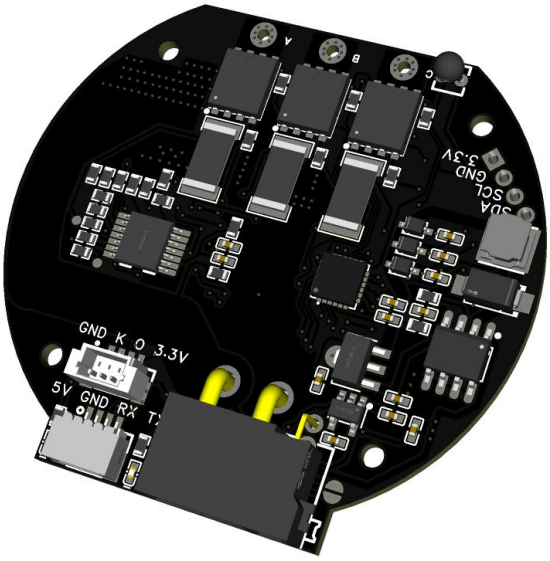
# FS6080-8 单编码电机驱动器手册

## 1. 电机驱动器简介




### 1.1. 主要特点

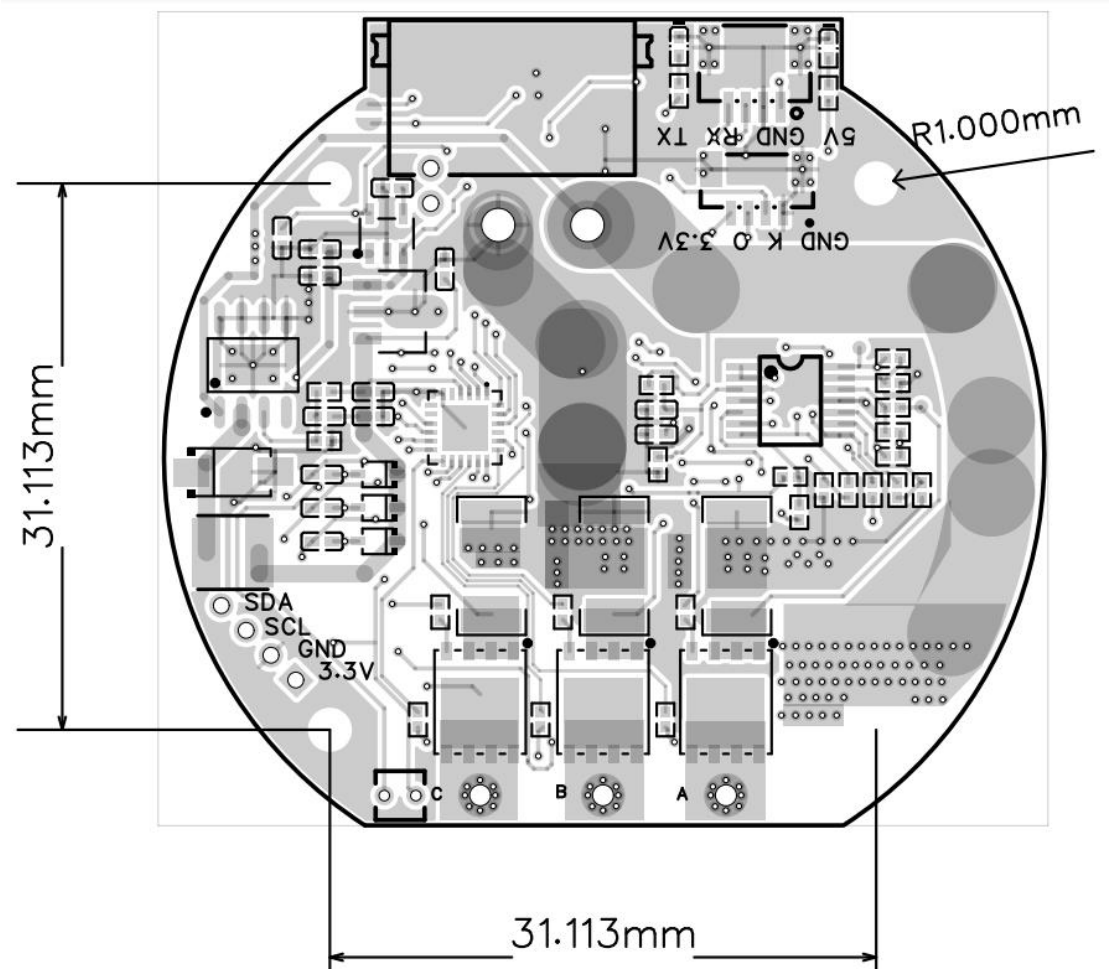
- 采用 FOC 控制方式
- 支持马达齿槽转矩脉动补偿
- 采用 6 颗 60V/170A MOSFET
- 采用 Cortex-M4 带硬件浮点运算器主控
- 支持磁编码器和感应磁铁同心度偏移补偿
- 支持自动测量电机相关参数并根据电机参数生成电流环控制增益
- 基于 12bit ADC 三相电流检测，相电流测量范围 -60 ~ +60A。
- 板载一颗分辨率高达 18bit 单圈绝对值磁编码器芯片，实现精确位置控制，无需每次上电校准电机编码器

## 1.2. 关键参数

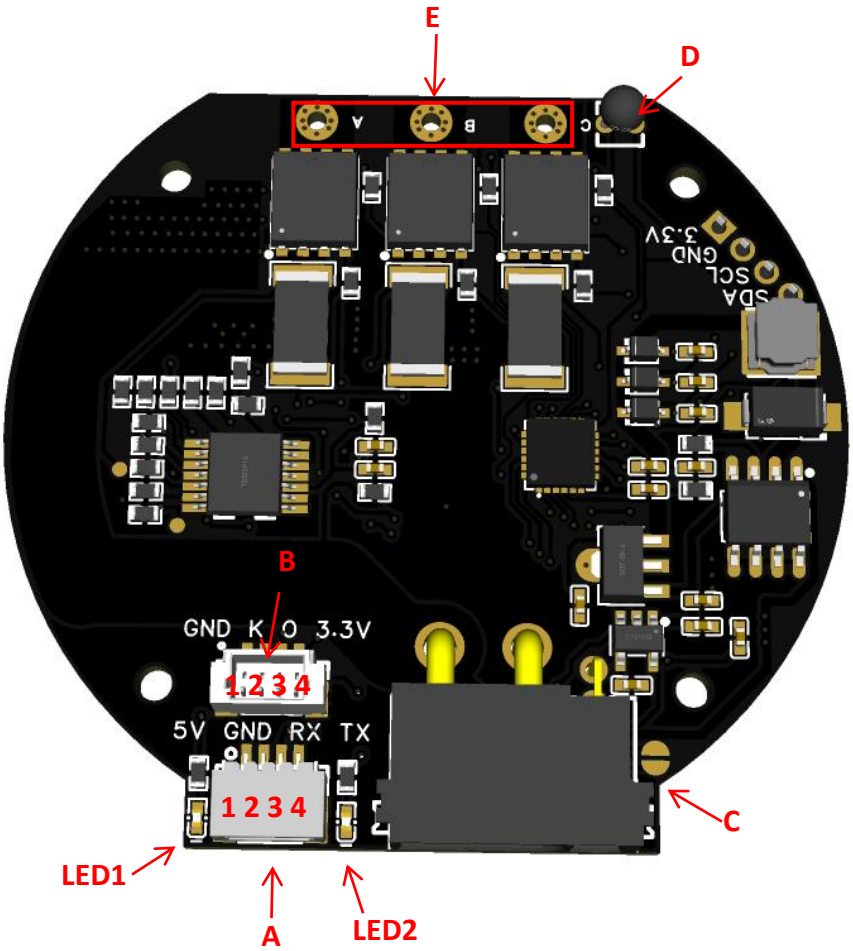
| | |
|---|---|
| 输入电压 | 24V |
| 输出电流 | 30ARMS |
| PWM频率 | 20KHz |
| 待机功耗 | 0.72W |
| 支持电机类型 | 三相永磁电机/BLDC/PMSM |
| 编码器类型 | 驱动板内部集成单圈绝对值磁编码器（分辨率 18 bit） |
| 通讯接口 | CAN总线 |
| 控制模式 | MIT控制 |

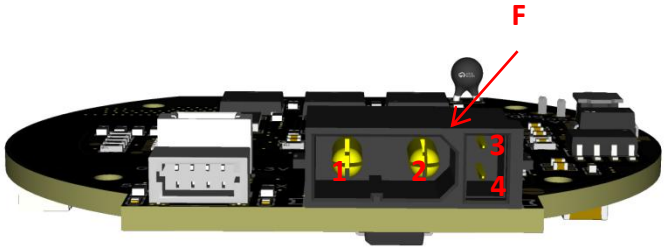
注意：以上参数非特殊说明即表示极限参数，超过将可能造成驱动器损坏！

## 1.3. 机械尺寸



## 1.4. 接线端口说明




### 1.4.1. A 串口

SH1.0 4P 卧贴耐高温端子

| 引脚 | 功能 |
| --- | --- |
| 1 | 5V(调试不用连接) |
| 2 | 数字信号地 |
| 3 | 串口 RX |
| 4 | 串口 TX |

### 1.4.2. B 烧录程序接口

SH1.0 4P 立贴耐高温端子

| 引脚 | 功能 |
| --- | --- |
| 1 | 数字信号地 |
| 2 | SWCLK |
| 3 | SWDIO |
| 4 | 3.3V |

### 1.4.3. C 终端电阻选择端(CAN)

默认为断开状态，即不启用板载CAN总线120Ω电阻，如果需要启用CAN终端电阻使用焊锡短路此处即可.

### 1.4.4. D 热敏电阻

电机线圈热敏电阻接口

### 1.4.5. E 电机接口

外径2.5mm 内径1.2mm 焊盘(双面)，相邻焊盘中心间距 7mm

### 1.4.6. F 电源

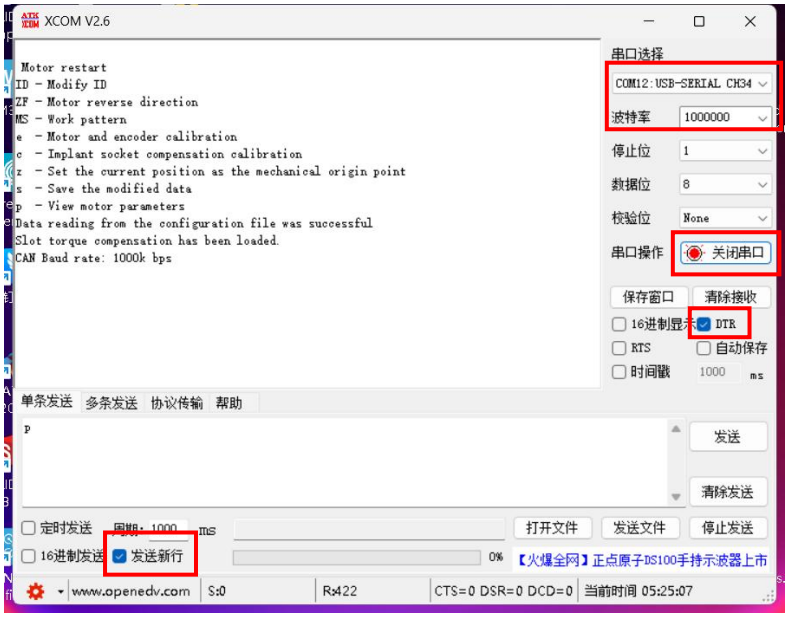
| 引脚 | 功能 |
| --- | --- |
| 1 | 电源正极 (24v) |
| 2 | 电源负极 |
| 3 | CAN H |
| 4 | CAN L |

### 1.4.7. LED

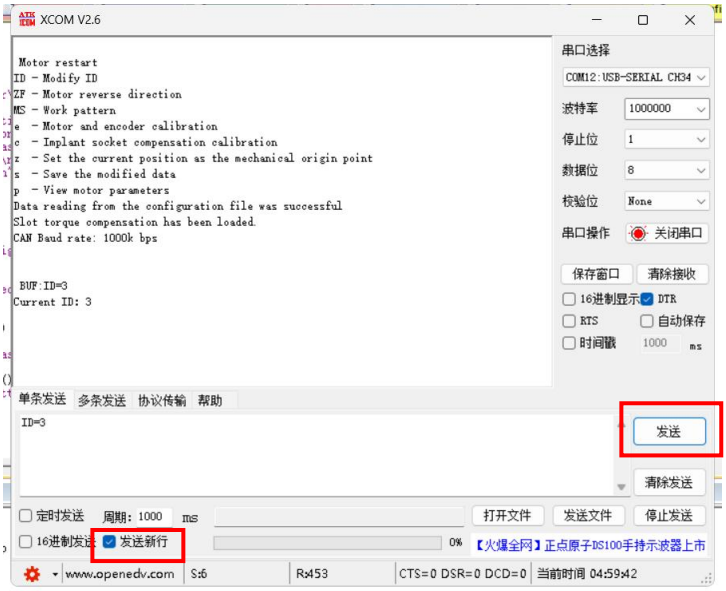
接通电源后驱动板上的 LED2 是电源指示灯将保持常亮，LED1 指示灯开始闪烁，LED1 指示灯通过不同的闪烁频率来指示当前驱动器工作状态。

# 2. 软件调试工具

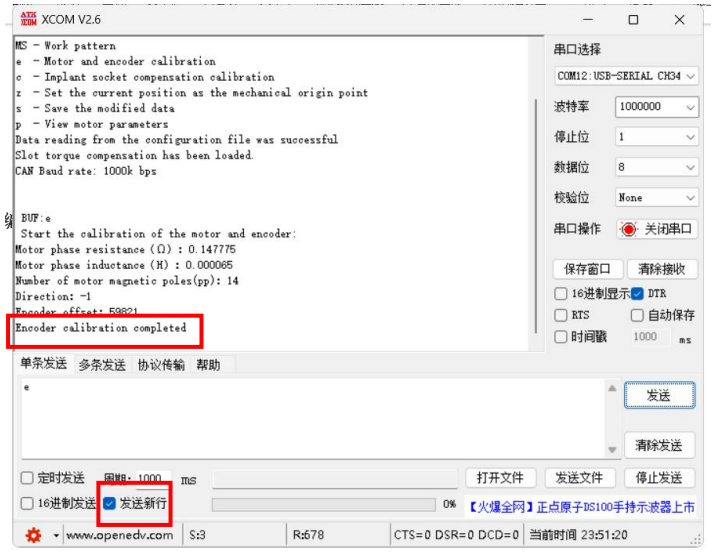
- 驱动板 ID 配置，编码器校准，零位设置等参数设置需要用到串口。可以使用 XCOM V2.6。（已经打包在附件）
- USB 转 TTL 通过 SH1.0 4P 线连接到驱动板的串口，插上 usb 转 TTL 到电脑。打开 XCOM V2.6 串口调试助手，选择正确的串口，并设置串口波特率为 1000000 等如下图所示。对驱动板通 24V 电源，XCOM V2.6 串口调试助手能正常打印如下内容，可以根据提示输入对应的命令。



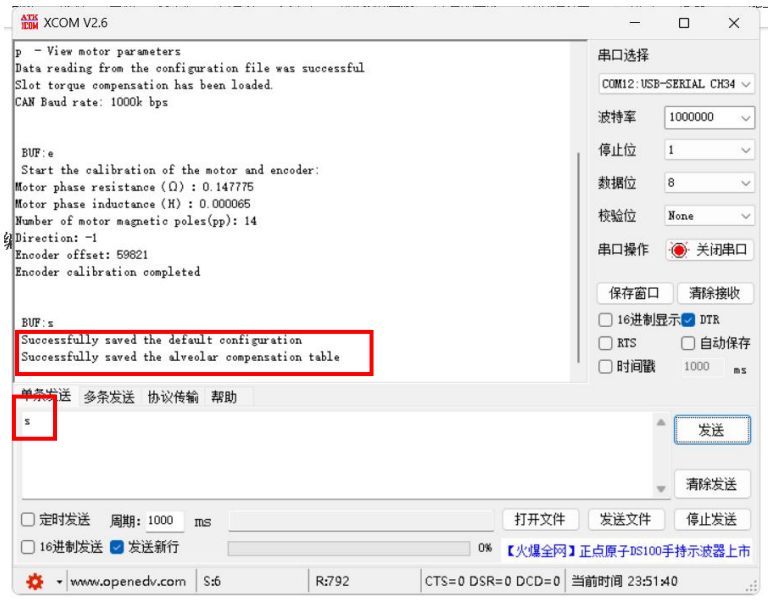

- 设置 ID 如下图所示，把 ID 设置成 3。输入：“ID=3” 并发送。



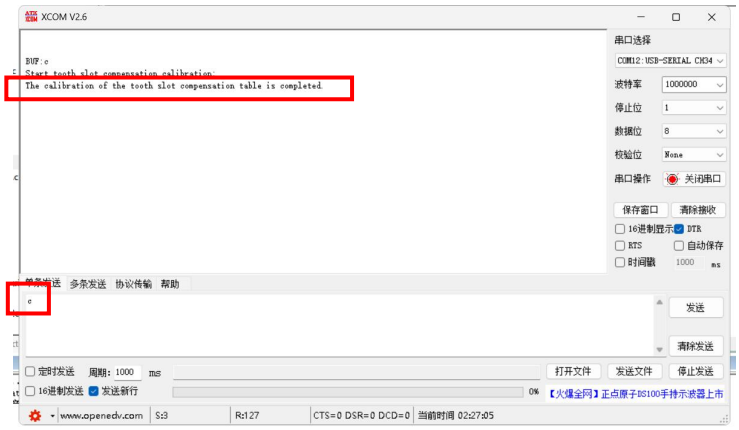

- 校准电机与编码器如下图所示，输入：“e”并发送。电机输出轴会匀速正转与反转，如果校准时出现卡顿或者不动立即关掉电机电源。校准成功会打印：Encoder calibration completed




- 保存电机参数如下图所示，输入：“s”并发送。打印下面数据表示保存数据成功，修改过数据或者校准过电机都要保存数据并重新上电。



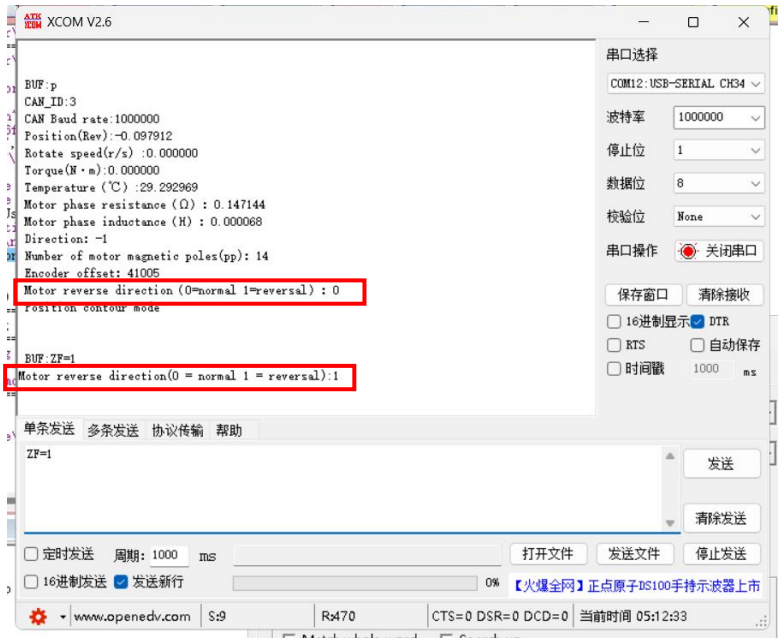
- 电机齿槽转矩补偿，输入：“c”并发送。电机会超低速匀速旋转，校准时间较长。校准完成后打印完成如下图（注意要先校准电机与编码器后再校准电机齿槽转矩补偿）




- 查看电机参数，输入：“p”并发送。(电机参数：ID、CAN 波特率、电机转子位置（转数）、电机转速等)

- 设置当前电机位置为原点，输入：“z”并发送。

- 设置电机旋转反向，输入：“ZF=1”并发送。（0：电机默认方向 1：电机旋转反向）




# 3. CAN 总线通讯协议

## 3.1. 帧格式

驱动器使用 CAN 总线标准 11 位 波特率 1M 消息标识数据帧和用户控制器进行通讯，标识符说明如下：

| CAN 标识符说明 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| bit 10 | bit 9 | bit 8 | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 |
| dir | node_id | | | | | cmd_id | | | | |

| 数据 | | | | | | | |
|---|---|---|---|---|---|---|---|
| Data 8 | Data 7 | Data 6 | Data 5 | Data 4 | Data 3 | Data 2 | Data 1 |

dir：通讯方向标志，控制器->电机：该位清零，电机->控制器：该位置位。
node_id： 电机的节点 ID 。
cmd_id：命令ID，标识消息类型，详细见下面命令说明。
通讯形式为客户端 - 服务器模型，电机作为服务器，用户控制器作为客户端，每次通讯由客户端(用户控制器)发起，服务器(电机)返回。不过也存在一些 cmd_id由电机主动报送给用户控制器，如：当dmg驱动器内部产生错误信息或状态信息改变时就会主动发送对应的cmd_id，这类命令一般以_REPORT结尾，如：CAN_CMD_MOTOR_DISABLE。

# 4. 命令说明

## CAN_CMD_MOTOR_DISABLE

停止电机，进入待机模式

- 用户控制器发送：
  - dir:0
  - cmd_id:0

| CAN 标识符 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| bit 10 | bit 9 | bit 8 | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 |
| 0 | node_id | | | | | 0 | | | | |

| 发送数据长度：0 | | | | | | | |
|---|---|---|---|---|---|---|---|
| ~~Data 8~~ | ~~Data 7~~ | ~~Data 6~~ | ~~Data 5~~ | ~~Data 4~~ | ~~Data 3~~ | ~~Data 2~~ | ~~Data 1~~ |

- 电机返回：
  - dir:1
  - cmd_id:0
  - Data 0~3:(int32)执行结果，正确执行为：0

| CAN 标识符 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| bit 10 | bit 9 | bit 8 | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 |
| 1 | node_id | | | | | 0 | | | | |

| 接收数据长度：4 | | | | | | | |
|---|---|---|---|---|---|---|---|
| ~~Data 8~~ | ~~Data 7~~ | ~~Data 6~~ | ~~Data 5~~ | Data 4 | Data 3 | Data 2 | Data 1 |
| | | | | (int32)执行结果，正确执行为：0 | | | |

## CAN_CMD_MOTOR_ENABLE

启动电机，进入工作模式

- 用户控制器发送：
  - dir:0
  - cmd_id:1

| CAN 标识符 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| bit 10 | bit 9 | bit 8 | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 |
| 0 | node_id | | | | | 1 | | | | |

| 发送数据长度：0 | | | | | | | |
|---|---|---|---|---|---|---|---|
| ~~Data 8~~ | ~~Data 7~~ | ~~Data 6~~ | ~~Data 5~~ | ~~Data 4~~ | ~~Data 3~~ | ~~Data 2~~ | ~~Data 1~~ |

- 电机返回：
  - dir:1
  - cmd_id:1
  - Data 0~3:(int32)执行结果，正确执行为：0

| CAN 标识符 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| bit 10 | bit 9 | bit 8 | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 |
| 1 | node_id | | | | | 1 | | | | |

| 接收数据长度：4 | | | | | | | |
|---|---|---|---|---|---|---|---|
| ~~Data 8~~ | ~~Data 7~~ | ~~Data 6~~ | ~~Data 5~~ | Data 4 | Data 3 | Data 2 | Data 1 |
| | | | | (int32)执行结果，正确执行为：0 | | | |

## CAN_CMD_SET_HOME

### 把当前位置设置为原点

- 用户控制器发送：
  - dir:0
  - cmd_id:11

| CAN 标识符 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| bit 10 | bit 9 | bit 8 | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 |
| 0 | node_id | | | | | 11 | | | | |

| 发送数据长度：0 | | | | | | | |
|---|---|---|---|---|---|---|---|
| ~~Data 8~~ | ~~Data 7~~ | ~~Data 6~~ | ~~Data 5~~ | ~~Data 4~~ | ~~Data 3~~ | ~~Data 2~~ | ~~Data 1~~ |

- 电机返回：

- dir:1
- cmd_id:11
- Data 0~3:(int32)执行结果，正确执行为：0

| CAN 标识符 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| bit 10 | bit 9 | bit 8 | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 |
| 1 | node_id | | | | | 11 | | | | |

| 接收数据长度：4 | | | | | | | |
|---|---|---|---|---|---|---|---|
| ~~Data 8~~ | ~~Data 7~~ | ~~Data 6~~ | ~~Data 5~~ | Data 4 | Data 3 | Data 2 | Data 1 |
| | | | | (int32)执行结果，正确执行为：0 | | | |

## CAN_CMD_ERROR_RESET

清除错误代码（电机过流保护、欠压保护、过压保护等标志）

- 用户控制器发送：
  - dir:0
  - cmd_id:12

| CAN 标识符 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| bit 10 | bit 9 | bit 8 | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 |
| 0 | node_id | | | | | 12 | | | | |

| 发送数据长度：0 | | | | | | | |
|---|---|---|---|---|---|---|---|
| ~~Data 8~~ | ~~Data 7~~ | ~~Data 6~~ | ~~Data 5~~ | ~~Data 4~~ | ~~Data 3~~ | ~~Data 2~~ | ~~Data 1~~ |

- 电机返回：
  - dir:1
  - cmd_id:12
  - Data 0~3:(int32)执行结果，正确执行为：0

| CAN 标识符 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| bit 10 | bit 9 | bit 8 | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 |
| 1 | node_id | | | | | 12 | | | | |

| 接收数据长度：4 |
|---|

| ~~Data 8~~ | ~~Data 7~~ | ~~Data 6~~ | ~~Data 5~~ | Data 4 | Data 3 | Data 2 | Data 1 |
|---|---|---|---|---|---|---|---|
| | | | | (int32)执行结果，正确执行为：0 | | | |

# CAN_CMD_GET_STATUSWORD

## 获取当前状态和错误信息

- 用户控制器发送：
  - dir:0
  - cmd_id:13

| CAN 标识符 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| bit 10 | bit 9 | bit 8 | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 |
| 0 | node_id | | | | | 13 | | | | |

| 发送数据长度：0 | | | | | | | |
|---|---|---|---|---|---|---|---|
| ~~Data 8~~ | ~~Data 7~~ | ~~Data 6~~ | ~~Data 5~~ | ~~Data 4~~ | ~~Data 3~~ | ~~Data 2~~ | ~~Data 1~~ |

- 电机返回：
  - dir:1
  - cmd_id:13
  - Data 0~7:(uint32)状态信息 ，(uint32)错误信息

| CAN 标识符 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| bit 10 | bit 9 | bit 8 | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 |
| 1 | node_id | | | | | 13 | | | | |

| 接收数据长度：8 | | | | | | | |
|---|---|---|---|---|---|---|---|
| Data 8 | Data 7 | Data 6 | Data 5 | Data 4 | Data 3 | Data 2 | Data 1 |
| (uint32)错误信息：<br>bit 0：驱动板启动ADC自检错误(无法通过CAN_CMD_ERROR_RESET清除，需要重启)<br>bit 1：编码器离线错误(无法通过CAN_CMD_ERROR_RESET清除，需要重启)<br>bit 2~15：保留<br>bit 16：过压保护<br>bit 17：欠压保护<br>bit 18：过流保护<br>bit 19~31：保留 | | | | (uint32)状态信息：<br>bit 0：电机使能<br>bit 1：目标到达<br>bit 2：电流限制<br>bit 3~31：保留 | | | |

# CAN_CMD_SET_CONFIG

## 设置MIT控制模式

- 用户控制器发送：
  - dir:0
  - cmd_id:17
  - Data 0~7:（int32）11 ，（int32）4 。

| CAN 标识符 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| bit 10 | bit 9 | bit 8 | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 |
| 0 | node_id | | | | | 17 （设置特定配置参数） | | | | |

| 发送数据长度：8 | | | | | | | |
|---|---|---|---|---|---|---|---|
| Data 8 | Data 7 | Data 6 | Data 5 | Data 4 | Data 3 | Data 2 | Data 1 |
| （int32）4 ：MIT控制模式<br>（0:转矩 1:转速 2：位置过滤波 3：轮廓位置） | | | | （int32）11 ：设置电机工作模式 | | | |

- 电机返回：
  - dir:1
  - cmd_id:17
  - Data 0~7:（int32）11 ，（int32）4 。

| CAN 标识符 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| bit 10 | bit 9 | bit 8 | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 |
| 1 | node_id | | | | | 17 | | | | |

| 接收数据长度：8 | | | | | | | |
|---|---|---|---|---|---|---|---|
| Data 8 | Data 7 | Data 6 | Data 5 | Data 4 | Data 3 | Data 2 | Data 1 |
| （int32）4 | | | | （int32）11 | | | |

## 设置电机方向反转

- 用户控制器发送：
  - dir:0
  - cmd_id:17
  - Data 0~7:（int32）1，（int32）1 。

| CAN 标识符 |
|---|
| |

| bit 10 | bit 9 | bit 8 | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | node_id | | | | | 17 （设置特定配置参数） | | | | |

| 发送数据长度：8 | | | | | | | |
|---|---|---|---|---|---|---|---|
| Data 8 | Data 7 | Data 6 | Data 5 | Data 4 | Data 3 | Data 2 | Data 1 |
| （int32）0 ：默认旋转方向<br>（int32）1 ：方向反转 | | | | （int32）1：设置带电机方向反转 | | | |

- 电机返回：
  - dir:1
  - cmd_id:17
  - Data 0~7:（int32）1，（int32）1。

| CAN 标识符 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| bit 10 | bit 9 | bit 8 | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 |
| 1 | node_id | | | | | 17 | | | | |

| 接收数据长度：8 | | | | | | | |
|---|---|---|---|---|---|---|---|
| Data 8 | Data 7 | Data 6 | Data 5 | Data 4 | Data 3 | Data 2 | Data 1 |
| （int32）1 | | | | （int32）1 | | | |

## CAN_CMD_SET_TORQUE_MIT

### 发送MIT力控数据给电机

- 用户控制器发送：
  - dir:0
  - cmd_id:11
  - Data 0~7:

| CAN 标识符 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| bit 10 | bit 9 | bit 8 | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 |
| 0 | node_id | | | | | 11 | | | | |

| 发送数据长度：8 | | | | | | | |
|---|---|---|---|---|---|---|---|
| Data 8 | Data 7 | Data 6 | Data 5 | Data 4 | Data 3 | Data 2 | Data 1 |

```
Data 0: [position[15-8]]
Data 1: [position[7-0]]
Data 2: [velocity[11-4]]
Data 3: [velocity[3-0], kp[11-8]]
Data 4: [kp[7-0]]
Data 5: [kd[11-4]]
Data 6: [kd[3-0], torque[11-8]]
Data 7: [torque[7-0]]

16 bit 位置命令, 范围：-16 至 16圈 （减速前的位置）
12 bit 速度命令, 范围： -50 至 50 r/s （减速前的速度）
12 bit kp, between 0 and 500 N-m/r
12 bit kd, between 0 and 5 N-m*s/r
12 bit feed forward torque, between -12and 12 N-m
```

- 电机返回：
  - dir:1
  - cmd_id:11
  - Data 0~7

| CAN 标识符 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| bit 10 | bit 9 | bit 8 | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 |
| 1 | node_id | | | | | 11 | | | | |

| 接收数据长度：8 | | | | | | | |
|---|---|---|---|---|---|---|---|
| Data 8 | Data 7 | Data 6 | Data 5 | Data 4 | Data 3 | Data 2 | Data 1 |

```
位置（圈）
position = uint16_to_float(data[0]<<8|data[1],MIN_P,MAX_P,16);//-16 至 16圈 （减速前的位置）
速度（r/s）
Velocity = uint16_to_float(data[2]<<8|data[3],V_MIN,V_MAX,16) + 0.00076294;//-50 至 50 r/s （减速前的速度）
扭矩（N-m）
torque   = uint16_to_float(data[4]<<8|data[5],T_MIN,T_MAX,16);// (-12Nm~12Nm)
温度（℃）
temp     = uint16_to_float((data[6]<<8|data[7]),MIN_T,MAX_T,16);//摄氏度

16 bit 位置命令, 范围：-16 至 16圈 （减速前的位置）
16 bit 速度命令, 范围： -50 至 50 r/s （减速前的速度）
16 bit feed forward torque, between -12and 12 N-m
16 bit 温度, 范围：-20 至 200 摄氏度

注意由于数据类型转换导致 电机不转时会有-0.00076294r/s 的速度偏差需要减掉
```

```
/**
 * @brief          发送浮点转16位数据
 * @param[in]      x:浮点
 * @param[in]      x_min:对应参数下限
 * @param[in]      x_max:对应参数上限
 * @param[in]      bits:参数位数
```

```
  * @retval         返回浮点值
  */
static int float_to_uint(float x, float x_min, float x_max, int bits)
{
  float span = x_max - x_min;
  float offset = x_min;
  if(x > x_max) x=x_max;
  else if(x < x_min) x= x_min;
  return (int) ((x-offset)*((float)((1<<bits)-1))/span);
}
```

```
/**
  * @brief          电机回文16位数据转浮点
  * @param[in]      x:16位回文
  * @param[in]      x_min:对应参数下限
  * @param[in]      x_max:对应参数上限
  * @param[in]      bits:参数位数
  * @retval         返回浮点值
  */
static float uint16_to_float(uint16_t x,float x_min,float x_max,int bits)
{
    uint32_t span = (1 << bits) - 1;
    float offset = x_max - x_min;
    return offset * x / span + x_min;
}
```

```
// CAN Command Packet Structure
// 16 bit 位置命令, 范围：-16 至 16圈 （减速前的位置）
// 12 bit 速度命令, 范围： -50 至 50 r/s （减速前的速度）
// 12 bit kp, between 0 and 500 N-m/r
// 12 bit kd, between 0 and 5 N-m*s/r
// 12 bit feed forward torque, between -12and 12 N-m
// CAN Packet is 8 8-bit words
// Formatted as follows.  For each quantity, bit 0 is LSB
// Data 0: [position[15-8]]
// Data 1: [position[7-0]]
// Data 2: [velocity[11-4]]
// Data 3: [velocity[3-0], kp[11-8]]
// Data 4: [kp[7-0]]
// Data 5: [kd[11-4]]
// Data 6: [kd[3-0], torque[11-8]]
// Data 7: [torque[7-0]]
void pack_cmd(uint8_t* msg, float p_des, float v_des, float kp, float kd, float t_ff)
{
     /// limit data to be within bounds ///
     p_des = fminf(fmaxf(P_MIN, p_des), P_MAX);
     v_des = fminf(fmaxf(V_MIN, v_des), V_MAX);
     kp    = fminf(fmaxf(KP_MIN, kp), KP_MAX);
     kd    = fminf(fmaxf(KD_MIN, kd), KD_MAX);
     t_ff  = fminf(fmaxf(T_MIN, t_ff), T_MAX);
     /// convert floats to unsigned ints ///
     int p_int = float_to_uint(p_des, P_MIN, P_MAX, 16);
```

```
    int v_int = float_to_uint(v_des, V_MIN, V_MAX, 12);
    int kp_int = float_to_uint(kp, KP_MIN, KP_MAX, 12);
    int kd_int = float_to_uint(kd, KD_MIN, KD_MAX, 12);
    int t_int = float_to_uint(t_ff, T_MIN, T_MAX, 12);
    /// pack ints into the can buffer ///
    msg[0] = p_int>>8;
    msg[1] = p_int&0xFF;
    msg[2] = v_int>>4;
    msg[3] = ((v_int&0xF)<<4)|(kp_int>>8);
    msg[4] = kp_int&0xFF;
    msg[5] = kd_int>>4;
    msg[6] = ((kd_int&0xF)<<4)|(t_int>>8);
    msg[7] = t_int&0xff;
}
```